Progressive Retrieval Attention for llama.cpp: Portable Sequence-Attached Semantic K/V in the GGUF Ecosystem

Jorge Simão

September 2026

Abstract

We ask for the narrowest real Native Memory primitive that Progressive Retrieval Attention (PRA) can exploit in llama.cpp. llama.cpp makes quantized local inference portable across CPUs and accelerator backends, while PRA asks for a different form of portability: independently addressable semantic records whose selected native key/value (K/V) states can be reused without becoming visible prompt tokens. We pin llama.cpp commit `458681e1d5d4a29a1463c4732e03226cf384b997`, implement a PRA SDK adapter, audit public slot and state APIs, and qualify selected-text execution on Apple M4 Pro and M5 CPU/Metal backends. A Qwen3-8B replication on 15 natural-QA examples used 10.0–42.2% of the full-context prompt tokens. Metal median time to first token fell on HotpotQA (696 to 457 ms) and QASPER (2733 to 348 ms), while 2Wiki was confounded by warm prefix reuse. A live slot checkpoint saved 13 positional tokens as 160,572 bytes and restored them in 2.327 ms. The audit shows that this state is sequential prefix state, not detached semantic K/V. The public memory API also exposes a native seam: encode a resource under one sequence and attach its existing unified-cache cells to a request with `llama_memory_seq_cp`. We wire that seam into `llama-server` with negotiated capabilities, pinned resource slots, query-only requests, and explicit release. All 25 matched E0/E2 generations and 25 warm reuses are token-exact; 15/15 concurrent requests share one resource correctly. Warm E2 latency is 0.898× E0, while cold attachment is 1.096× E0. Thus the pinned server path qualifies a prefix-positioned E2 mechanism, while scheduler-owned resource allocation remains E3 work.

1 Introduction

The companion runtime paper defines the common E0/E2/E3 and evidence contracts [1]. This paper isolates the llama.cpp mechanism and its portability boundary.

Local inference has two kinds of context pressure. The first is computational: long prompts increase prefill work and K/V residency. The second is semantic: applications need records, documents, tool outputs, and task state that do not naturally form one immutable prefix. llama.cpp handles the first problem with portable GGUF execution, quantization, slots, and cache controls. PRA addresses the second by separating a logical record from the small region selected for a particular query.

The integration question is not whether a cache can be saved. It is whether an engine can attend to selected K/V from an independently named resource while preserving ordinary query positions, masks, and a single normalization over direct and memory keys. Confusing a restored conversation prefix with such memory would produce an attractive but incorrect E2 claim.

This paper contributes: (i) a capability-honest llama.cpp SDK provider and adapter; (ii) a pinned audit of public server and cache-state seams; (iii) live CPU and Metal E0 qualification with frozen evidence selections; and (iv) a minimal native selected-K/V implementation over llama.cpp's public C API with schedule-matched logit parity; and (v) a live server protocol with resource lifecycle, warm reuse, absent-memory controls, and shared-resource concurrency.

2 PRA Contract

For query state q and selected native memory (K_M, V_M), native PRA requires one attention normalization,

$$\mathrm{Attn}(q) = \mathrm{softmax}\left(\frac{q[K_D; K_M]^\top}{\sqrt{d}} + M\right)[V_D; V_M], \tag{1}$$

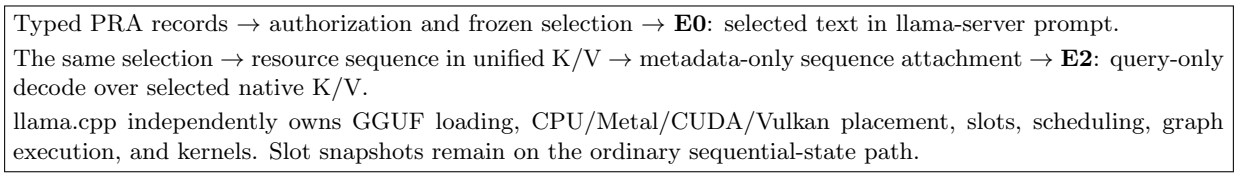

Figure 1: Ownership and integration boundary. Both E0 and the narrow E2 path are now measured through HTTP; the in-process probe independently checks logits.

where (K_D, V_D) is direct sequential context and M encodes the correct causal and resource mask. Concatenating decoded memory text is E0. Restoring a saved sequential cache is prefix reuse. Neither is native E2.

We use four evidence levels:

E0 selected source appears as ordinary visible text;

E1 logical resource identity survives transport, but attention remains text/prefix based;

E2 the model consumes detached selected native K/V;

E3 the engine scheduler owns PRA placement, prefetch, sharing, and eviction.

3 Pinned llama.cpp Audit

The pinned source was built with CMake 4.4.3 and Ninja 1.13.2. The resulting `llama-server` exposes OpenAI-compatible chat, explicit slots, unified KV configuration, cache quantization, CPU execution, and Metal offload. GGUF owns model topology and quantized weights; llama.cpp owns tensor placement, graph execution, attention kernels, and sequential cache representation.

The public slot save/restore endpoint is operational only when the server is launched with `--slot-save-path`. Its object is an engine slot: token positions, recurrent conversational state, and ordinary cached K/V. It does not accept an immutable resource identifier, per-layer selected intervals, or separate direct/memory geometry. Consequently, slot state cannot be used to claim E2.

The in-process C API has a distinct seam. `llama_memory_seq_cp` adds one sequence's cache cells to another sequence. With `kv_unified=true`, the pinned implementation updates cell metadata and does not copy K/V buffers. A selected resource can therefore remain under a stable resource sequence, be attached to a request sequence, and be consumed without resubmitting its text. This is native transport with ordinary prefix-equivalent positions; it is not yet an arbitrary-position memory branch.

4 Implementation

`LlamaCppEngineAdapter` exposes the OpenAI-compatible path and reports E0. An optional `LlamaCppSlotClient` derives checkpoint filenames from the model fingerprint, tenant, session, resource URI/version, and source hash. This prevents an application from reusing a slot solely because two visible prompts look alike. The filename discipline improves prefix-state isolation but does not upgrade its integration level.

Only an explicit `LlamaCppNativeExecutor` can advertise E2. The concrete `LlamaCppNativeServerExecutor` first negotiates `/pra/capabilities`. It encodes selected text once in a resource slot marked `pra_pin_resource`, sends only the query to a separate request slot, and names the source with `pra_resource_slot`. At launch, the server rejects non-unified caches, busy or empty sources, source/destination aliasing, multimodal tokens, LoRA mismatch, and context overflow. It strips a duplicate query BOS, clones logical prompt metadata, and calls `seq_cp(source,destination)`. The unified cache adds destination membership to existing cells rather than copying K/V payloads.

Pinned resource slots are excluded from ordinary LRU selection, idle-slot clearing, and emergency idle eviction. This rule was required by a live warm reuse failure found during development: upstream unified-cache housekeeping correctly clears ordinary idle slots, but a PRA resource is deliberately idle between queries. `DELETE /pra/resources/{slot}` removes sequence membership, clears the pin, and returns the slot to the ordinary pool. The current SDK serializes one explicit resource/request slot pair; allocation, authorization, and multiplexing of many pairs belong to an E3 scheduler.

Table 1: Metal E0 qualification. Prompt ratio is selected/full. TTFT is median milliseconds; F1 is selected/full.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.410	37.0 / 60.1	0.314 / 0.314
QASPER	0.375	31.0 / 58.9	0.243 / 0.077
2Wiki	0.517	35.6 / 27.3	0.029 / 0.129

Table 2: CPU backend smoke test. The three-case cells are calibration evidence, not workload-scale estimates.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.411	305.5 / 530.5	0.103 / 0.190
QASPER	0.356	225.7 / 534.6	0.070 / 0.106
2Wiki	0.525	256.8 / 275.5	0.000 / 0.000

5 Experimental Method

We used an Apple M4 Pro with 14 CPU cores, 20 GPU cores, and 48 GB unified memory. The model was Qwen2.5-0.5B-Instruct in Q4_K_M GGUF. Metal used `-ngl 99`; CPU used `-ngl 0`. Both used a 4096-token context and four slots. The natural benchmark freezes the selected evidence before engine execution and compares no context, E0 selected text (up to 384 source tokens), and full context (up to 1024 source tokens). Metal covers five examples from each of HotpotQA, QASPER, and 2WikiMultiHopQA with two repeats. CPU is a three-example-per-dataset backend smoke test.

Quality is token F1 on short generated answers. System metrics are measured from streaming HTTP responses. The small model, short generation cap, and small cohort make quality directional rather than conclusive. CPU and Metal cohorts also differ in repeat count; their absolute quality is not paired.

We then ran a larger-model replication on an Apple M5 with 10 CPU cores, 10 GPU cores, and 16 GB unified memory. It used the official Qwen3-8B Q4_K_M GGUF with the official llama.app binary (build 10679, commit `50f068fff`). Selection was again frozen before execution. The Metal cohort used five unique examples per dataset, two repeats, a 384-token selected cap, and a 4096-token full-context cap. A one-example-per-dataset CPU run with four threads is reported only as a backend diagnostic; it is not a matched throughput comparison.

The native mechanism cohort uses pinned llama.cpp commit `458681e1d5d`, Qwen2.5-0.5B-Instruct Q4_K_M, and five factual prompts. Each prompt runs on CPU and Metal. The E0 control encodes resource and query in two calls under one request sequence. E2 encodes the same resource tokens under a resource sequence, attaches them to the request in unified-cache mode, and decodes only the same query tokens. We compare every vocabulary logit at the first prediction and through four greedy decode steps. A one-call FULL control is retained separately because changing prefill batch shape can introduce ordinary floating-point differences.

The server cohort uses the same 0.5B GGUF on the 48-GB M4 Pro Metal backend. Five factual resource/query pairs are repeated five times. For every pair, E0 receives selected text plus query, whereas E2 first pins the selected resource and then sends only the same query. We also repeat the E2 attachment without re-encoding, run a query with no resource, and issue three concurrent requests against one immutable source. Reported request latency is client-observed HTTP wall time; this short non-streaming run does not separate TTFT from completion.

6 Results

The robust result is input reduction, not universal quality or latency improvement. Selected prompts occupy 35.6–52.5% of the full prompt across the two backend runs. Metal selected-text TTFT improves for HotpotQA and QASPER, but 2Wiki reverses direction because prefix-cache state and a short full prompt dominate this small run. Selected evidence preserves HotpotQA F1 on Metal and raises QASPER F1, while 2Wiki loses quality. This is consistent with an upstream selection limitation, not evidence for or against native representation.

The 8B replication strengthens the systems observation without turning the smoke cohort into a quality claim. Selected text reduced the visible prompt by 57.8–90.0%. It reduced median TTFT by 34.3% on HotpotQA and 87.3% on QASPER. The reversed 2Wiki median coincides with 682 mean cached tokens for the full condition versus 207 for selected text; its full-context p95 was still 1931 ms versus 860 ms selected. Thus warm prefix reuse can dominate a tiny median even while selected text improves the tail. Quality was approximately preserved on

Table 3: Qwen3-8B Q4_K_M replication on M5 Metal (five examples per dataset, two repeats). Prompt ratio is selected/full; TTFT is p50 milliseconds.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.327	457 / 696	0.381 / 0.181
QASPER	0.100	348 / 2733	0.185 / 0.150
2Wiki	0.422	452 / 247	0.253 / 0.255

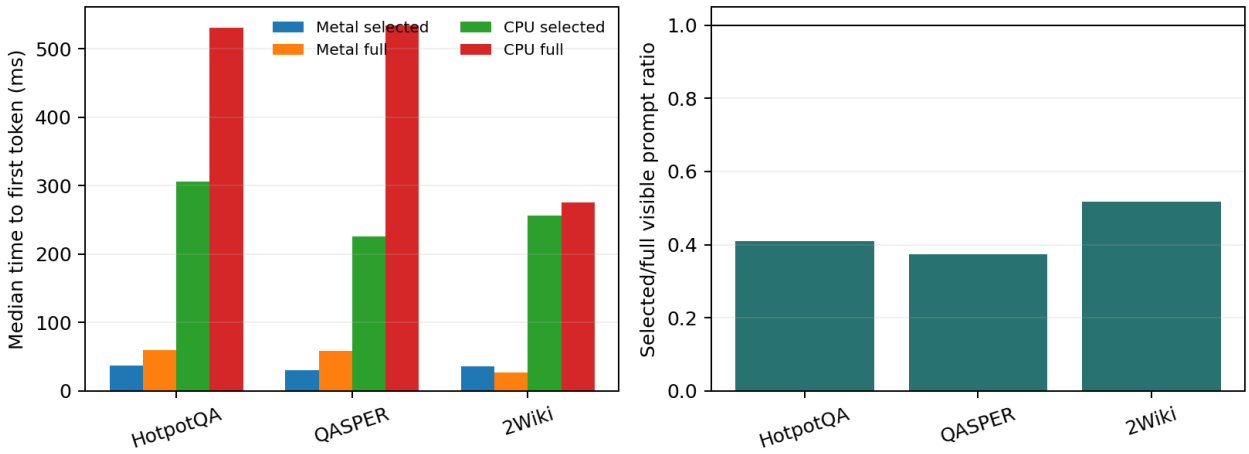

Figure 2: Measured E0 context-pressure qualification. Selected text reliably reduces visible tokens; latency and quality effects remain workload dependent.

2Wiki, improved on these five Hotpot examples, and changed only slightly on QASPER. These signs are descriptive because examples, not repeated requests, are the independent units.

The four-thread CPU diagnostic was orders of magnitude slower: selected TTFT was roughly 37–39 s and full-context TTFT 45–68 s across the three cases, whereas Metal produced about 24 output tokens/s. This establishes accelerator placement as operationally necessary for this 8B local workload; the CPU sample is too small for a backend-quality comparison.

6.1 Slot-state qualification

A live slot operation saved 13 cached tokens, wrote 160,572 bytes in 0.412 ms, and restored the same 13 tokens from 160,572 bytes in 2.327 ms. This is useful for repeated sequential sessions. It also demonstrates why slots are not a substitute for PRA: the saved object is much larger than a logical reference, is position-shaped, and has no selected-resource attention contract.

6.2 Native sequence-attachment qualification

Table 4: Pinned in-process native selected-K/V mechanism cohort. Exact logits refer to schedule-matched split E0 versus E2 through four decode steps.

Backend	Cases	Exact logits	Exact decode	Isolation	Warm reuse
CPU	5	5/5	5/5	5/5 exact	5/5
Metal	5	5/5	5/5	5/5 $< .006$	5/5

Schedule-matched E0 and E2 have maximum logit error exactly zero in all 10 runs, including every persistent-decode step. One-shot FULL retains the same top token in 10/10 runs but differs numerically from split E0/E2: maximum FULL–E2 error ranges up to 0.415 on CPU and 0.0201 on Metal. The split control localizes that difference to prefill batch shape rather than native sequence attachment. In unified-cache mode, upstream `seq_cp`

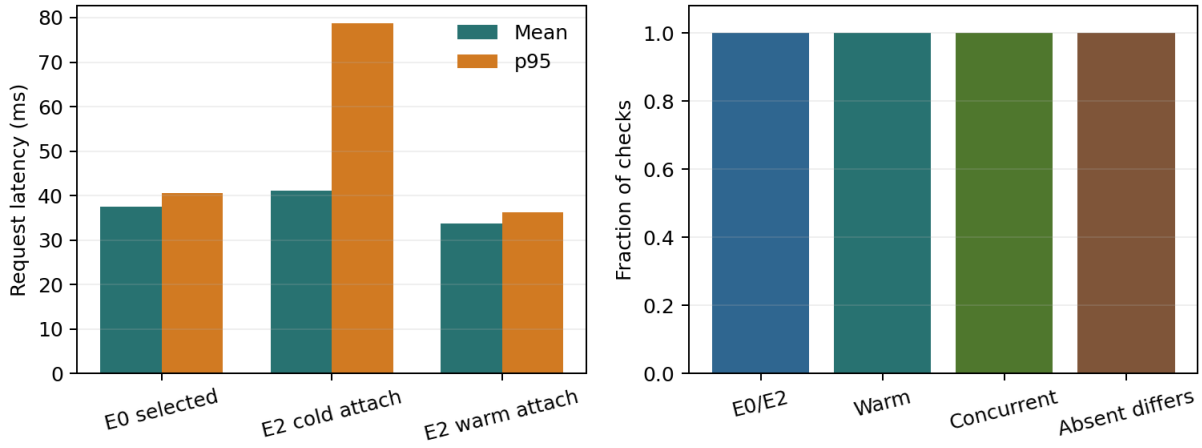

Figure 3: Server-level native attachment. Warm E2 is faster than E0 in this short cohort; cold E2 retains a first-use tail. All parity, reuse, concurrent sharing, and absent-memory checks pass.

adds the request sequence ID to existing cells; it does not copy their physical K/V payload. After first-request cleanup, absent second requests match a fresh query exactly on CPU. Metal preserves all five top tokens but has maximum logit differences of 0.0015–0.0058 because resident masked cells change the unified-cache kernel shape. Explicitly reattaching the still-resident resource reproduces first-request logits exactly in all 10 runs.

6.3 Native server lifecycle and economics

Table 5: Matched llama-server E0/E2 cohort on M4 Pro Metal. Latencies are milliseconds over 25 requests; ratios use means.

Condition	Mean	p95	New/wire tokens	Exact vs E0
E0 selected text	37.46	40.51	17.6	reference
E2 cold attach	41.05	78.76	5.4	25/25
E2 warm attach	33.65	36.29	5.4	25/25

E0 evaluates 17.6 prompt tokens on average. E2 carries 5.4 query tokens on the wire and attaches 12.2 already encoded native tokens; initial resource ingest costs 8.00 ms on average. Cold E2 is 1.096× E0 mean latency, but warm E2 falls to 0.898× E0. The p95 difference is larger in the cold condition because first-use graph work is included. The result therefore supports the expected economic boundary: one-shot attachment need not win, whereas reuse can amortize ingestion and avoid selected-text prefill.

All 25 E0/E2 token sequences match exactly, as do all 25 warm repeats. All 25 no-resource controls differ from their resource-enabled counterparts. Three simultaneous request slots share one pinned resource with 15/15 exact outputs, no physical K/V copy, and mean throughput of 48.62 requests/s. Explicit release removes the source sequence and clears its pin. These checks close transport, reuse, isolation, sharing, and cleanup for this narrow mechanism; they do not establish natural-QA quality or a production scheduler.

7 Result Interpretation and Stop Gate

The mechanism gate is closed at narrow scope: E0 works, slot state is precisely classified, unified-cache sequence attachment has exact CPU/Metal logits, and the server path now has explicit ingest, attach, reuse, concurrency, isolation, and release. Multi-resource controls, physical byte accounting under sustained load, CUDA/Vulkan replication, and E3 scheduler ownership remain open.

Table 6: Qualification by backend and level at the pinned revision.

Backend	E0	E2	E3
CPU	measured (0.5B; 8B diagnostic)	5-case in-process mechanism	not implemented
Metal	measured (0.5B, 8B)	25-run server + 5-case logits	not implemented
CUDA	not run here	not run	not run
Vulkan	not run here	not run	not run

8 Negative Results and Next Work

The cohorts remain small; the 8B replication broadens model scale but not the number of independent questions. Native parity uses five synthetic factual prompts and one model, not natural QA. The concurrency cohort has only 15 requests, and non-streaming responses do not provide TTFT, ITL, or reliable tail estimates. We do not yet measure gold-answer log probability, physical bytes saved under pressure, or multi-resource budgeting. The sequence-attached mechanism also preserves prefix-equivalent positions; arbitrary non-prefix geometry requires a graph-level attention extension. Next work is a tenant-aware slot allocator, multiple selected resources, longer natural-QA workloads, then matched CPU, Metal, CUDA, and Vulkan runs.

9 Conclusion

llama.cpp supplies a portable selected-text baseline and cheap sequential slot persistence. Slot restore remains E0/E1. The pinned public C API nevertheless supports a narrow E2 path: unified-cache sequence membership can expose an already encoded selected resource to a query-only server request without a physical K/V copy. Exact matched generation, warm reuse, shared-resource concurrency, absent-memory isolation, and explicit cleanup now work end to end. Warm E2 beats selected text in this short cohort, while cold E2 does not. The remaining product boundary is no longer basic transport; it is E3 allocation, authorization, multi-resource geometry, pressure behavior, and portability.

Artifact Availability

The adapter, tests, raw JSON, figure generator, commands, and manuscript are available on the Paper 6.7 research branch. The README records the exact branch, pinned model, backend flags, and benchmark invocation.

References

[1] J. Simao. *Productizing Progressive Retrieval Attention: A Runtime, Gateway, and Deployment Architecture.* 2026.

[2] G. Gerganov et al. *llama.cpp.* 2023–2026. https://github.com/ggml-org/llama.cpp

[3] llama.cpp contributors. *GGUF format documentation.* https://github.com/ggml-org/ggml/blob/master/docs/gguf.md

[4] Qwen Team. *Qwen2.5 Technical Report.* 2024. https://arxiv.org/abs/2412.15115